

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - BACKEND API RESTful (SCRUM-22)

PATRÓN ARQUITECTÓNICO SELECCIONADO

Se adopta una arquitectura de Microservicio Modular con separación en capas (Layered Architecture) siguiendo el patrón Repository-Service-Controller. Esto garantiza bajo acoplamiento, alta cohesión y facilidad de pruebas unitarias mediante inyección de dependencias.

STACK TECNOLÓGICO

- Runtime: Node.js con Express.js (framework minimalista, alto rendimiento, ecosistema maduro para APIs RESTful)
- Base de datos: SQLite (via better-sqlite3) con soporte en memoria para testing
- Autenticación: JWT (jsonwebtoken) + bcrypt para hashing de contraseñas
- Caché: Redis (ioredis) para consultas frecuentes de usuarios
- Mensajería: RabbitMQ (amqplib) para eventos CRUD asincrónicos
- Validación: Joi o express-validator para validación estricta de payloads
- Documentación: Swagger/OpenAPI 3.0 (swagger-ui-express + swagger-jsdoc)
- Testing: Jest + Supertest para pruebas unitarias e integración con mocking de DB
- Containerización: Docker multi-stage con imagen base Alpine
- CI/CD: GitLab CI (.gitlab-ci.yml) con stages Test, Security_Scan y Build

DECISIONES DE SEGURIDAD (DevSecOps)

1. Autenticación JWT: Tokens firmados con HS256, expiración configurable (default 1h), almacenados en header Authorization Bearer.
2. Autorización RBAC: Middleware de verificación de rol 'admin' en todos los endpoints de gestión de usuarios.
3. Encriptación: bcrypt con salt rounds ≥ 12 para contraseñas. Nunca se almacena ni retorna la contraseña en texto plano.
4. HTTPS: Configurado a nivel de infraestructura (reverse proxy Nginx/Kubernetes Ingress con TLS termination).
5. Validación de entrada: Sanitización y validación estricta de todos los campos antes de procesamiento.
6. Rate Limiting: express-rate-limit para prevenir ataques de fuerza bruta y DDoS.
7. Helmet.js: Headers HTTP de seguridad (CSP, HSTS, X-Frame-Options, etc.).
8. CORS: Configuración restrictiva de orígenes permitidos.
9. Variables de entorno: Secrets gestionados via .env (nunca hardcoded), con dotenv.
10. SAST: Integración de Semgrep o Bandit equivalente (eslint-plugin-security) en pipeline CI.
11. Dependency Scan: npm audit + Snyk en stage Security_Scan del pipeline.

ENDPOINTS PRINCIPALES

- POST /api/v1/auth/login -> Autenticación, retorna JWT
- POST /api/v1/users -> Crear usuario (admin)
- GET /api/v1/users -> Listar usuarios (admin)
- GET /api/v1/users/:id -> Obtener usuario por ID (admin)
- PUT /api/v1/users/:id -> Actualizar usuario (admin)
- DELETE /api/v1/users/:id -> Eliminar usuario (admin)
- POST /api/v1/deployments -> Registrar despliegue (admin)
- GET /api/v1/metrics/average-time/:id_app -> Tiempo promedio SUCCESS últimos 7 días
- POST /api/v1/sync/users -> Sincronización con servicios externos
- GET /api/health -> Health check

CAPAS DE LA APLICACIÓN

1. Routes Layer: Definición de rutas y binding de middlewares
2. Middleware Layer: Auth JWT, RBAC, validación, rate-limit, logging
3. Controller Layer: Manejo de request/response HTTP, delegación a servicios
4. Service Layer: Lógica de negocio, orquestación, publicación de eventos

5. Repository Layer: Abstracción de acceso a datos (SQLite)
6. Cache Layer: Interceptor Redis para consultas frecuentes
7. Events Layer: Publicador/consumidor RabbitMQ para sincronización

ESCALABILIDAD Y RENDIMIENTO

- Arquitectura stateless: Permite escalabilidad horizontal con múltiples instancias
- Redis caché: Reduce carga en SQLite para GET de usuarios frecuentes
- RabbitMQ: Desacopla operaciones síncronas de notificaciones a otros módulos
- Docker + Kubernetes: Orquestación con HPA (Horizontal Pod Autoscaler) basado en CPU/memoria
- Load Balancer: Nginx o Kubernetes Service distribuye tráfico entre pods
- Connection pooling: Gestión eficiente de conexiones a DB y Redis

PIPELINE CI/CD (GitLab CI)

- Stage TEST: Jest con cobertura mínima 80%, reporte JUnit XML
- Stage SECURITY_SCAN: npm audit, Semgrep SAST, Trivy para imagen Docker
- Stage BUILD: Docker build multi-stage, push a registry (solo si stages previos exitosos)
- Artifacts: Reportes de cobertura y seguridad almacenados como artifacts de GitLab

DOCKERFILE MULTI-STAGE

- Stage 1 (builder): node:18-alpine, instalación de dependencias de producción
- Stage 2 (runtime): node:18-alpine limpio, copia solo artefactos necesarios
- Usuario sin privilegios: Creación de usuario 'appuser' (UID 1001), nunca root
- Optimización de capas: COPY package*.json primero para aprovechar caché de Docker

2. MÉTRICAS DE ITERACIÓN (QA)

- .gitlab-ci.yml | Estado: APROBADO | Iteraciones: 2
- Dockerfile | Estado: APROBADO | Iteraciones: 1
- .dockerignore | Estado: APROBADO | Iteraciones: 2
- .env.example | Estado: APROBADO | Iteraciones: 1
- .eslintrc.js | Estado: APROBADO | Iteraciones: 1
- .gitignore | Estado: APROBADO | Iteraciones: 1
- package.json | Estado: APROBADO | Iteraciones: 1
- jest.config.js | Estado: WARNING (Forzado) | Iteraciones: 3
- docs/openapi.yaml | Estado: APROBADO | Iteraciones: 1
- src/app.js | Estado: APROBADO | Iteraciones: 1
- src/server.js | Estado: APROBADO | Iteraciones: 1
- src/config/database.js | Estado: APROBADO | Iteraciones: 1
- src/config/redis.js | Estado: APROBADO | Iteraciones: 1
- src/config/rabbitmq.js | Estado: APROBADO | Iteraciones: 1
- src/config/env.js | Estado: APROBADO | Iteraciones: 1
- src/routes/index.js | Estado: APROBADO | Iteraciones: 1
- src/routes/auth.routes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/users.routes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/deployments.routes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/metrics.routes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/sync.routes.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/auth.controller.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/users.controller.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/deployments.controller.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/metrics.controller.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/sync.controller.js | Estado: APROBADO | Iteraciones: 1
- src/services/auth.service.js | Estado: APROBADO | Iteraciones: 1
- src/services/users.service.js | Estado: APROBADO | Iteraciones: 1
- src/services/deployments.service.js | Estado: APROBADO | Iteraciones: 1
- src/services/metrics.service.js | Estado: APROBADO | Iteraciones: 1
- src/services/sync.service.js | Estado: APROBADO | Iteraciones: 1
- src/repositories/base.repository.js | Estado: APROBADO | Iteraciones: 1

- src/repositories/users.repository.js | Estado: APROBADO | Iteraciones: 1
- src/repositories/deployments.repository.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/auth.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/rbac.middleware.js | Estado: APROBADO | Iteraciones: 3
- src/middlewares/validate.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/rateLimiter.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/errorHandler.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/logger.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/validators/user.validator.js | Estado: APROBADO | Iteraciones: 1
- src/validators/deployment.validator.js | Estado: APROBADO | Iteraciones: 1
- src/validators/auth.validator.js | Estado: APROBADO | Iteraciones: 1
- src/models/user.model.js | Estado: APROBADO | Iteraciones: 1
- src/models/deployment.model.js | Estado: APROBADO | Iteraciones: 1
- src/events/publisher.js | Estado: APROBADO | Iteraciones: 1
- src/events/consumers/userEvents.consumer.js | Estado: APROBADO | Iteraciones: 1
- src/cache/users.cache.js | Estado: APROBADO | Iteraciones: 1
- src/migrations/001_create_users_table.js | Estado: APROBADO | Iteraciones: 1
- src/migrations/002_create_deployments_table.js | Estado: APROBADO | Iteraciones: 1
- src/utils/jwt.utils.js | Estado: APROBADO | Iteraciones: 2
- src/utils/bcrypt.utils.js | Estado: APROBADO | Iteraciones: 1
- src/utils/response.utils.js | Estado: APROBADO | Iteraciones: 1
- src/utils/logger.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/controllers/users.controller.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/controllers/deployments.controller.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/controllers/metrics.controller.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/services/users.service.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/services/deployments.service.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/services/metrics.service.test.js | Estado: APROBADO | Iteraciones: 2
- tests/unit/utils/jwt.utils.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/utils/bcrypt.utils.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/users.integration.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/deployments.integration.test.js | Estado: APROBADO | Iteraciones: 2
- tests/mocks/database.mock.js | Estado: WARNING (Forzado) | Iteraciones: 3
- tests/mocks/redis.mock.js | Estado: APROBADO | Iteraciones: 1
- tests/mocks/rabbitmq.mock.js | Estado: APROBADO | Iteraciones: 1